PRACTICAL 10

Aim :

Implement asynchronous background processing using Node.js native EventEmitter. When a task is created or deleted, emit an event handled by a notification listener without blocking the main API response.

Code :

backend/events.js :-

```
const EventEmitter = require('events');

class TaskEvents extends EventEmitter {}

module.exports = new TaskEvents();
```

backend/listeners.js :-

```
const taskEvents = require('./events');

function delay(milliseconds) {
  return new Promise((resolve) => setTimeout(resolve, milliseconds));
}

taskEvents.on('task-created', (task) => {
  setImmediate(async () => {
    try {
      await delay(2000);
      console.log(`[Notification] Task "${task.title}" created at ${new Date().toISOString()}`);
    } catch (error) {
      taskEvents.emit('error', error);
    }
  });
});

taskEvents.on('task-deleted', (task) => {
  setImmediate(() => {
```

```
    console.log(`[Notification] Task "${task.title}" deleted at ${new Date().toISOString()}`);
  });
});
taskEvents.on('error', (error) => {
  console.error(`[Notification error] ${error.message}`);
});
```

Code :

backend/routes/tasks.js event usage :-

```
const taskEvents = require('../events');

router.post('/', asyncHandler(async (req, res) => {
  const task = await Task.create(req.body);
  cache.invalidateTaskCaches();
  res.status(201).json(task);
  taskEvents.emit('task-created', task);
}));

router.delete('/:id', ensureValidId, asyncHandler(async (req, res) => {
  const task = await Task.findByIdAndDelete(req.params.id);
  if (!task) return res.status(404).json({ error: 'Task not found' });
  cache.invalidateTaskCaches(req.params.id);
  res.status(200).json(task);
  taskEvents.emit('task-deleted', task);
}));
```

backend/server.js listener registration :-

```
const taskRoutes = require('./routes/tasks');
const authRoutes = require('./routes/auth');
require('./listeners');
```

Event flow :

```
POST /api/tasks
      |
      v
Save task to MongoDB
      |
      v
```

Send 201 response immediately

|

v

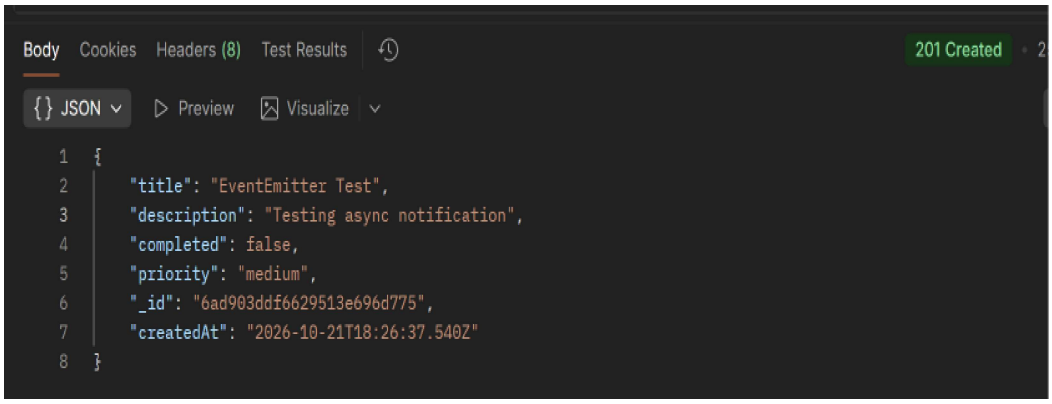
Emit task-created event

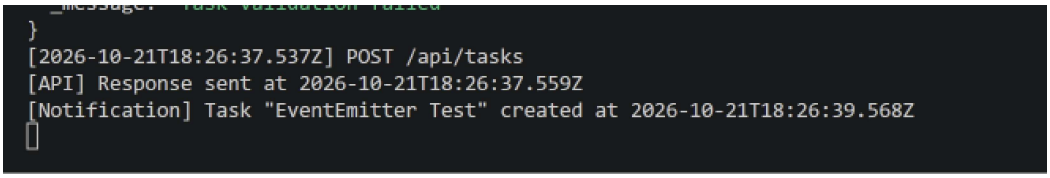
|

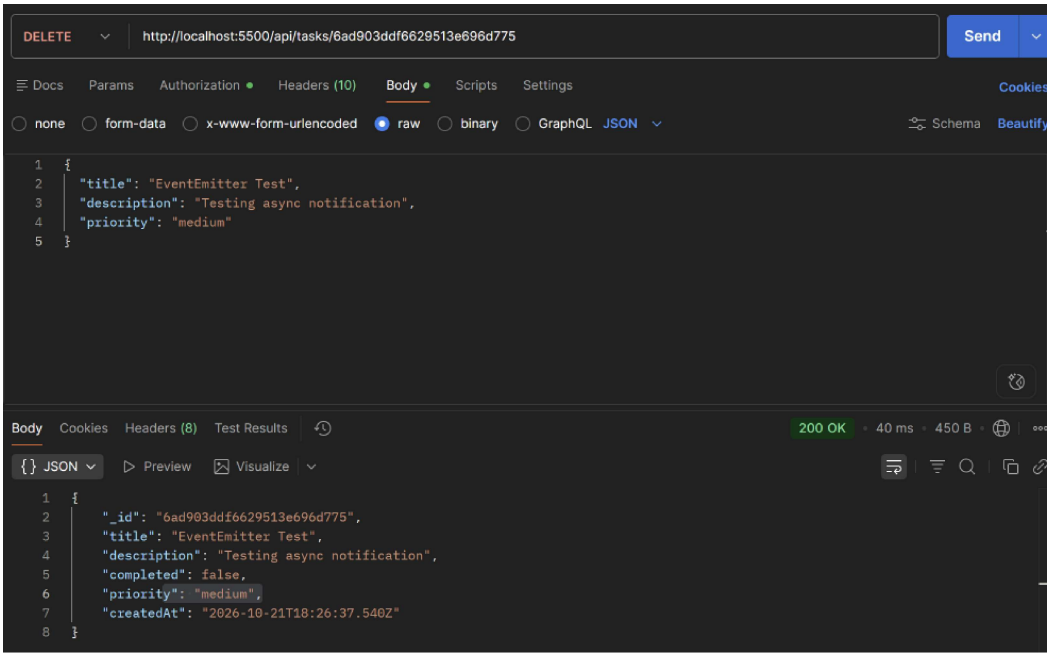
v

Delayed notification listener

Output :







```
[nodemon] starting `node server.js`
MongoDB connected
Server running on port 5500
[2026-10-21T18:28:13.689Z] DELETE /api/tasks/6ad903ddf6629513e696d775
```

Result :

An EventEmitter-based asynchronous notification system was successfully implemented. Task-created and task-deleted events are emitted after the API response, and listeners process notifications without blocking the request-response cycle. The error listener handles notification failures safely.